

I Ralph-looped Opus overnight. It reduced my local model switching with cold backfilling context of 135k+ on llama.cpp from ~165s -> 5s! TL;DR - USE SLOTS!

Research

#TL;DR - Opus Ralph-looped on shortening my cold-start back-fill on restoring chats with large contexts. It Cherry-picked two open llama.cpp PRs (#20819 + #20822 by @European-tech) plus built a Python supervisor that hashes normalized prefixes and hardlinks slot bins on NVMe.

Result: KV cache survives model swaps on a single 3090 Ti, dropping per-session swap overhead from several mins to as little as 5s from cold to RESULT response. Restore is 160–800ms regardless of model.

Requires byte-compatible KV across runs and `OPENCODE_EXPERIMENTAL_CACHE_STABILIZATION=1` to keep opencode's system prompt stable. Both PRs still unmerged.

I now have what genuinely feels like a near full Claude Code experience locally via opencode albeit not frontier models.

#####

First my new build stack, which I've been polishing for the last 10 days...

- Ryzen 9950x
- Single RTX 3090 Ti (24GB)
- 96GB DDR5 Samsung 9100
- 2TB Gen5 NVMe. and other irrelevant bits

I am running a 7-step Council-Build-Council pipeline:

Spec > Review > Plan > Build > Code Review > Security Review > UAT Review

Chair

- Qwen3.6-27B orchestrator, 200k context.

Builders

- Qwen3-coder-30B (tested, benchmarked, outperformed qwen3.6 on my codebase)

Reviewers, Councillors and the "wtf is wrong with this, debug brainstorm" models.

- gemma-4-31b
- gpt-oss-20b
- qwen3.6-27b
- nemotron-cascade-2-30b
- qwen3.6-35b
- qwen3-coder-30b

Tiny council. Uber fast 20 sec, parallel critiques before big council.

- minstral-8b
- nemotron-nano-4b
- qwen3-4b

Yes, Opus wrote the below. Yes, I proof-read it. Nope, I'm not sorry I made Opus write it :-)

#####

Single GPU = all models serialize through one slot. Parallel dispatch from the chair's POV; llama-swap actually executes them one at a time.

I wanted to get as close to claude code locally as possible however without persistent KV cache, every model entry pays full prefill against its own context. Old news for most here probably, but being new to LLM locally this was news to me, and VERY annoying. So swap times ...

- Chair Qwen3.6 holds 130K -> ~165s prefill on every return.
- Reviewers hold ~20K -> ~30s.
- Coders hold ~50k -> ~60s.

Across spec critique + 3-builder fanout + review + security review + UAT + 2-3 remediation cycles, that's ~22 min of pure prefill overhead per session. Wasted. My existing workflow porting from Claude Code + Ollama Cloud appeared dead on arrival.

The options were I either just watch it all happen sequentially, stick to one model, try to reduce my cycles. ** OR ** set Opus on a Ralph loop overnight with all the access it wants to Sonnet and Ollama cloud to figure this out.

I chose the latter.

Two open PRs by @European-tech persist slot state across process death were the key:

- **#20819** - server: *persist context checkpoints across slot save/restore* - companion `<file>.checkpoints` file (magic `0x4C4C4350` "LLCP"). <https://github.com/ggml-org/llama.cpp/pull/20819>
- **#20822** - server: *auto-save/restore slot state in router mode* - `--auto-save-slots` / `--auto-restore-slots`. <https://github.com/ggml-org/llama.cpp/pull/20822>

Opus cherry-picked both then wrote a Python supervisor wrapping llama-server: hashes message prefixes, pokes `/slots/0?action=restore` before forwarding, hardlinks `<prefix_hash>.bin` <-> `<full_hash>.bin` so prefix-matching requests hit the cache via either key. Slot bins on Gen5 NVMe; Linux page cache acts as implicit RAM tier (96GB DDR5 keeps many bins hot, ~3GB/s effective restore speed).

Real per-model numbers (pulled from supervisor logs this morning):

```
# Chair (orch, 138K-token ctx) - two consecutive returns between coder dispatches:
RESTORE slot0 n_restored=138151 ms=801    ->  RESULT elapsed=4.7s
RESTORE slot0 n_restored=138301 ms=765    ->  RESULT elapsed=17.3s

# Reviewer (Gemma-31B, ~19K-token review ctx) swapping in/out across 3 review passes:
RESTORE slot0 n_restored=19293 ms=334     ->  RESULT elapsed=27.1s
RESTORE slot0 n_restored=19303 ms=651     ->  RESULT elapsed=27.0s
```

```
RESTORE slot0 n_restored=19293 ms=651 -> RESULT elapsed=27.9s
RESTORE slot0 n_restored=19472 ms=161 -> RESULT elapsed=64.3s
```

Restore is **160-800ms regardless of model**, scaling with KV size. Without slots, those would be ~30s prefill (Gemma 19K) and ~165s prefill (Qwen3.6 27B 138K) every time. Save-then-evict on swap-out is also ~1s, so **a full swap-cycle (out + in) is ~2s** across any model in the rotation.

I keep the gguf files in system memory for qwen3.6 and qwen3-coder.30b to allow for extremely quick cycles in the Chair orchestrator <> builder flows.

Pipeline cost breakdown for one session (chair + 3-builder fanout + reviewer + 3-way security fanout + UAT + 2 remediation cycles). Each row = a model entry. Chair-returns dominate because chair has 10x more ctx than workers.

Step	Without slots (prefill)	With slots (restore)
Spec fanout: 3 council members swap in/out sequentially	3 x ~30s = 90s	3 x ~2s = 6s
Chair-return after spec	165s	5s
Build fanout: 3 builders swap in/out sequentially (worktrees)	3 x ~30s = 90s	3 x ~2s = 6s
Chair-return after build merge	165s	5s
Reviewer (Gemma)	~30s	~2s
Chair-return after review	165s	5s
Security fanout: 3 reviewers swap in/out	3 x ~30s = 90s	3 x ~2s = 6s
Chair-return after security	165s	5s
UAT (builder runs tests)	~30s	~2s
Chair-return after UAT	165s	5s
Remediation x 2 (builder + chair-return each)	2 x (30+165) = 390s	2 x (2+5) = 14s
Total swap overhead	~22 min	~65s

(Generation time itself unchanged - slots only kill prefill.)

Tiny council (3 small models that co-resident in ~11GB VRAM as a non-swap llama-swap group) doesn't pay swap cost between members; they all stay loaded. Full 3-way critique runs in **19.4s end-to-end**. Re-entering chair after that is ~5s instead of ~165s.

Architecture sketch:

```
[Chair (orch)] --evict + save slot--> [Worker, llama-swap]
      ^                               |
      .                               .
```

```

      |                                     v
      | ~5s restore                        ~2s restore + gen + save
      |                                     |
+---- slot bin (NVMe) <----- saved here on swap-out
      ^
Linux page cache (RAM, ~96GB) holds hot bins

```

Caveats:

- KV must be byte-compatible across runs -> same model, same `--ctx-size`, same `-ctk/-ctv` quant, same arch flags. Change any -> invalidate bins.
- First-ever visit to a model still pays prefill (no slot exists). Slot reuse pays off from the 2nd visit onward - which is every visit in an iterative pipeline.
- Worth it only if you're both ctx-heavy AND swap-heavy. Single-model setups get nothing.

Both PRs still open. Load-bearing for any router-style multi-model setup. Would love to see them merged. Happy to share the supervisor wrapper.

#####

#####

Below is the full list of things Opus found and either worked around or incorporated along the way...

llama.cpp side

1. `/slots/N?action=save|restore` is in-process only — slot state evaporates when llama-swap kills the server (i.e. changes model).
2. PR #20819 alone insufficient — checkpoints saved to disk but no auto-restore on startup. Test image (PR #20819 only) still showed T2≈171s every tune.
3. PR #20822 is the load-bearing piece — `--auto-save-slots` / `--auto-restore-slots`. Adding it dropped T2 to 6.5s.
4. Both PRs still **open**, not merged. Both by @European-tech.
 - <https://github.com/ggml-org/llama.cpp/pull/20819>
 - <https://github.com/ggml-org/llama.cpp/pull/20822>
5. Build b9026 added strict `common_fit_params` abort — same args that fit pre-cherry2 (ctx 262144 + ngl 48 q4/q4) now fail with "cannot meet free memory target". Forced ctx drop 262144 → 196608 on coder.

Slot storage

6. tmpfs at /tmp blew the 30GB cap during tuning — moved slot dir to NVMe `/home/nick/tmp/llama-slots/`.

7. Linux page cache acts as implicit RAM tier in front of NVMe — restore measured ~3GB/s (page cache hit) vs ~1.5GB/s raw Gen5 sequential.

8. `<f>.bin.checkpoints` companion files orphan when `<f>.bin` evicted — added orphan-purge sweep to slot-cleanup.sh.

9. Unknown-model dirs (longctx, midctx, q3xl etc.) lingered after consolidation — added unknown-dir purge (recovered 30GB).
10. Edit-tool file overwrites create new inode → docker bind mount stale → ctr restart needed for `slot-supervisor.py` changes to take effect.
11. Symlinks for prefix-hash bins broke (host-path absolute target unresolvable) — switched to **hardlinks** (`os.link`) and paired `.bin` + `.bin.checkpoints`.

slot-supervisor.py wrapper

12. `cache_prompt: true` + `id_slot` must be force-injected into every request body.
13. Body must be normalized before hashing — opencode injects volatile fields (`<TS>`, `<DATE>`, `<EPOCH>`, `<CLOCK>` etc.). Without normalization, prefix hash flips every turn → 100% MISS.
14. `/metrics` endpoint blocks behind llama-server's task queue under load — added 5s background poll + cached body served on the fast path.
15. Read-only endpoint timeout reduced to 5s; `/v1/chat/completions` keeps 600s.
16. Prefix-hash and full-hash bins must coexist (one slot, two filenames) — hardlinks solve.

llama-swap

17. Bind-mounting config alone doesn't hot-reload — needs `-watch-config` flag.
18. `swap:false` + `exclusive:true` (tiny_council group) keeps small models co-resident; `swap:true` + `exclusive:true` (gpu_chat group) gives mutual eviction across the 24GB slot.

opencode-side cache instability (not our slot, but breaks our slot reuse)

19. opencode merges static + dynamic system content into one block → cache miss every turn (issues #5224, #20110).
20. Workaround flag exists: `OPENCODE_EXPERIMENTAL_CACHE_STABILIZATION=1` (PR #14743) — freezes date + instruction file reads for process lifetime.
21. Adding/removing skills changes system-prompt bytes → prefix hash flip → one-time MISS until next save. Expected, not a bug.

Related opencode tickets:

- PR #14743 — fix(cache): system split + tool stability + CACHE_STABILIZATION flag
- PR #20109 — narrower split-only fix

Production migration

22. Single-step Dockerfile build was incomplete — needed Dockerfile.proxy-cherry2 layered on `crucible-burnin:cherry2` to bundle llama-swap with cherry-pick'd llama-server.

23. Switching slot dir from /tmp → /home/nick/tmp required compose volume edit + container restart.

24. Test container 502s during burn-in iterations — production proxy held VRAM. Fixed by `docker stop crucible-proxy` in `run-iter.sh` trap.

Verification numbers (real run)

25. Chair-return: 138K-token KV restored in 801ms / 765ms; end-to-end 4.7s / 17.3s vs ~165s prefill without.

26. Reviewer (Gemma 19K ctx): restore 161–651ms; end-to-end 27–64s, dominated by generation, not prefill.

27. Tiny council (ministral + nemotron + qwen3-4b co-resident): full 3-way critique 19.4s end-to-end.

Pipeline overhead

28. Full Council-Build-Council session (spec fanout + 3 builders + review + security fanout + UAT + 2 remediation): swap overhead drops from ~22 min → ~65s.

16

5

[Join the conversation](#)

Sort by: **Best**

[Search Comments](#)

TheApadayo • 8h ago

I was messing around with something similar in llama-swap and hadn't gotten it to work yet. Can you get better swap performance by using /dev/shm to save the KV files? It's a ram disk which should be even faster than NVMe if you have the system RAM to spare. Basically gets around the "randomness" of files getting evicted from the OS file cache.

2

[Reply](#)



yes_i_tried_google [OP](#) • 8h ago • Edited 4h ago

It depends on your disk. With the Samsung 9100 the difference is <1 second so not worth the hassle. Actual restore time to KV (ignoring decoding a response) is <2 seconds.

The rest of the time I quoted in my post was response time, so normal decode tps. Sorry that wasn't clear in my post

2

[Reply](#)

nicksterling • 4h ago

Very interesting. Are you willing to share your llama-server wrapper script?

1

[Reply](#)

getstackfax • 10h ago

This is one of the most useful local-stack writeups I've seen around here nice....because it focuses on the actual bottleneck: not just tokens/sec, but context re-entry cost.

A lot of people compare local models as if the question is only:

"Which model is smartest?"

But in a multi-model coding pipeline, the real question becomes:

"Can the stack preserve enough state between model roles that orchestration does not collapse under prefill overhead?"

Your numbers make the practical point pretty clearly.

If every chair/reviewer/builder re-entry has to rebuild large context from cold, multi-model routing becomes painful even if each individual model is good.

With persistent slot restore, the workflow changes from:

model swap = minutes of dead time

to:

model swap = usable control-plane operation

That is a different category.

The caveats also seem important:

- byte-compatible KV only
- same model/config/context/quant assumptions
- first visit still pays prefill
- useful mainly for context-heavy + swap-heavy workflows
- unmerged PR dependency
- system prompt stability matters
- slot invalidation needs to be understood

So I would not frame this as "everyone should use slots."

I'd frame it as:

if you are building a local router-style coding workflow with repeated role handoffs, context persistence becomes part of the architecture.

The thing I'd want next is a run receipt per pipeline:

- which model acted as chair/builder/reviewer
- which context was restored
- cache hit/miss
- restore time
- generation time
- invalidation reason
- files touched

- tests run
- review result

That would make it easier to tell whether the pipeline is actually improving work, not just becoming faster.

Still, this is exactly the kind of infrastructure that makes local multi-model coding feel less like a science project and more like an actual workstation.

1

Reply